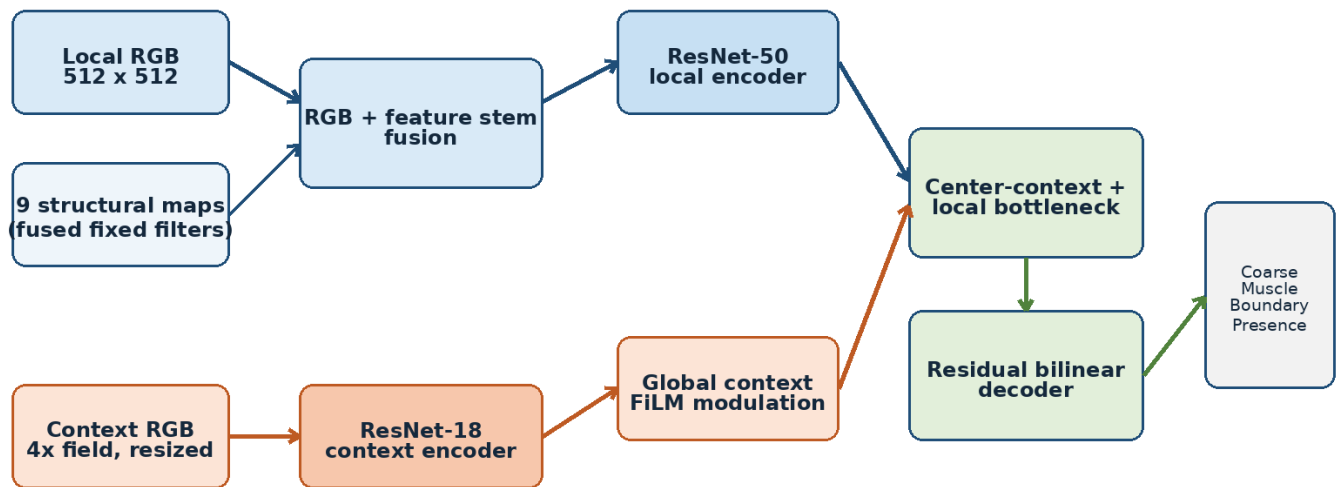


CNN-MalGen06

Project Walkthrough - Current Revision

Accuracy-first, dual-scale tissue segmentation in PyTorch

AccurateTissueNet - current data and model flow



Final probabilities are mutually exclusive and sum to one. The coarse parent probability is split into Fibrocartilage and Muscle by a conditional child head.

Current revision: derivative filters are fused into two convolution calls; optional cached structural maps can bypass per-batch recomputation.

Revision basis: repository master reviewed July 30, 2026

Includes VRAM caching, batched CUDA collation, optional feature caching, fused fixed filters, patch-level splitting, compile controls, TF32/fused AdamW, and throughput diagnostics.

1. Executive summary

CNN-MalGen06 performs six-class, native-resolution tissue segmentation. The model combines a detailed local view, a wider context view, and nine structural feature maps. Its hierarchical output represents Muscle conditionally inside a Fibrocartilage-or-Muscle parent class while preserving six mutually exclusive final probabilities.

What changed in the current revision

The project is no longer only an accuracy-first model with a conventional CPU DataLoader. It now includes a second, high-throughput CUDA-resident data path, fused fixed-filter computation, device-side metric accumulation, fused optimizer support, TF32, optional torch.compile, deterministic patch-level splits, cache-size estimation, and explicit performance/update-count diagnostics.

Current tissue classes

ID	Class	Role in hierarchy
0	Bone	Independent coarse class
1	Fibrocartilage	Child of Fibrocartilage-or-Muscle
2	Cartilage	Independent coarse class
3	Muscle	Conditional child of Fibrocartilage-or-Muscle
4	Marrow	Independent coarse class
5	Background	Independent coarse class

Revision highlights

- The fixed derivative bank is fused from eleven small convolutions into two larger calls: one multi-output derivative convolution and one grouped Gaussian smoothing convolution.
- Training images can be stored as uint8 CUDA tensors, with a prebuilt context pyramid, device masks, device boundary maps, cached sampling metadata, and optional FP16 structural maps.
- Cached samples are collated directly on the GPU in channels-last format. Geometry and color jitter are applied in groups or whole batches instead of through many per-sample kernels.
- Automatic validation defaults to a deterministic patch-level split when no separate validation directory is supplied. For final experiments, an external specimen/patient split remains recommended.
- CUDA training enables AMP, channels-last, cuDNN benchmarking, TF32, and fused AdamW by default. torch.compile is optional and reports steady-state throughput in patches/s.
- The local workstation observation improved from 9.3 to 20.5 patches/s at batch size 4 after compilation, approximately a 2.2x steady-state gain.

2. Contents and project map

Section	Topic
3	Repository and data contract
4	Data sampling and augmentation
5	CUDA cache and collation
6	Model architecture
7	Losses, metrics, and optimization
8	Validation splitting
9	Recommended training workflows
10	Performance measurement and tuning
11	Warnings and troubleshooting
12	Prediction and quantification
13	File-by-file reference
14	Revision checklist and suggested experiments

Repository files

File	Purpose
data.py	Discovers image/mask pairs, selects patch centers, builds local/context crops, manages CUDA caches, and returns augmented training samples.
model.py	Defines HandcraftedFeatures and AccurateTissueNet, including dual encoders, FiLM context modulation, residual decoder, and hierarchical output heads.
train.py	Defines losses, metrics, CUDA collation, batch augmentation, data splitting, optimizer/scheduler, checkpointing, compile controls, and throughput reporting.
predict.py	Runs overlapping native-resolution tiled inference with blending and writes class/probability outputs.
quantify.py	Converts segmentation outputs to hard-mask and probability-weighted physical areas.
smoke_test.py	Checks imports, tensor shapes, and forward/backward compatibility before a long run.
slurm_train.sbatch / slurm_predict.sbatch	Cluster job templates for training and prediction.
README.md	Concise operational reference; this walkthrough expands the design and tuning rationale.

3. Repository and data contract

The project expects matched source images and single-channel indexed masks. Source images retain native resolution; only sampled patches are resized or cropped.

```
dataset/  
  images/  
    specimen_001.tif  
    specimen_002.tif  
  masks/  
    specimen_001.png  
    specimen_002.png
```

- Image and mask stems must match.
- Image and mask dimensions must match exactly.
- Mask values must be class IDs 0 through 5; palette brightness must not replace indexed values.
- The default local crop is 512 x 512 pixels. `crop_size` must be a positive multiple of 32.
- The default `context_scale` is 4, so the context field covers approximately 2048 x 2048 native pixels before being represented as a 512 x 512 tensor.

Native-resolution principle

The model is trained and inferred on native-resolution tiles. Global resizing of the specimen would change tissue scale and area calibration, so the project instead uses local crops and overlapping tiled prediction.

Core dataset length

For a normal image-level dataset view, the logical number of patch samples is:

```
len(dataset) = number_of_selected_images * patches_per_image
```

For a patch-level split, a deterministic candidate pool is created first and Subset views assign individual patch IDs to training or validation.

4. Data sampling and augmentation

4.1 Patch-center selection

When training from a separate validation directory or an image-level split, patch centers are sampled from three strategies:

Strategy	Default probability	Purpose
Bone-Fibrocartilage boundary	0.45	Concentrates training on the clinically important interface and supports the dedicated boundary head.
Foreground-class focus	0.35	Selects a present non-background class and samples a center from that tissue.
Unbiased random	0.20	Maintains broad specimen coverage and background/context diversity.

Cached mode stores Bone-Fibro boundary coordinates and present-class metadata so repeated patch selection does not rescan every full-resolution mask. Class focus first uses bounded rejection sampling and falls back to an explicit coordinate search only when needed.

4.2 Deterministic patch pools

When the default patch-level automatic split is active, fixed patch centers are arranged on an aspect-ratio-aware near-square grid. This keeps patch ownership stable: a deterministic patch ID cannot appear in both training and validation.

Important tradeoff

Patch-level splitting disables boundary- and class-focused center selection because the candidate locations must remain fixed before the split. Training augmentation still applies after the assignment.

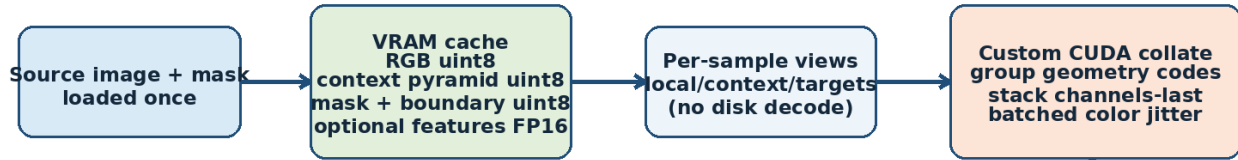
4.3 Paired augmentation

- Horizontal flip with probability 0.5.
- Vertical flip with probability 0.5.
- Rotation by 0, 90, 180, or 270 degrees.
- Brightness sampled from 0.85 to 1.15.
- Contrast sampled from 0.85 to 1.15.
- Saturation sampled from 0.75 to 1.25.
- Gamma sampled from 0.85 to 1.15.

Local RGB, context RGB, class mask, boundary target, and optionally cached structural maps receive the same geometric transformation. Photometric augmentation affects RGB. When handcrafted maps are precomputed, those maps continue to describe the original structural image while the RGB receives color jitter; this must be validated rather than assumed beneficial.

5. CUDA cache and batched collation

Cached training pipeline



Cached mode uses `num_workers=0` and `pin_memory=False` because the RGB, context, masks, and boundaries are already CUDA tensors.

The current collate function sorts samples by geometric transform code, applies each flip/rotation to a contiguous slice, and computes Muscle presence once for the batch.

5.1 What is stored

Cache item	Representation	Approximate cost
Source RGB	uint8 CUDA, CHW	3 bytes/source pixel
Context pyramid	uint8 CUDA, downsampled by <code>context_scale</code>	~0.19 bytes/source pixel at scale 4
Class mask	uint8 CUDA plus CPU copy	1 device byte/source pixel
Bone-Fibro boundary	uint8 CUDA	1 byte/source pixel
Boundary coordinates / present classes	CPU metadata	Data dependent; used for sampling
Optional structural maps	9-channel FP16 CUDA	18 bytes/source pixel

5.2 Why workers become zero

A conventional DataLoader uses subprocesses to decode files and create CPU tensors. In cached mode, Dataset items contain CUDA tensors and launch CUDA crop operations. The training loader therefore uses `num_workers=0`, `pin_memory=False`, and a custom CUDA collate function. Validation remains on the conventional CPU path and can still use normal workers and pinned memory.

```

train_workers = 0 # only when --cache-training-images is active on CUDA
pin_memory = False # RGB/context/targets are already on the device
collate_fn = device_channels_last_collate
  
```

5.3 Custom collation

- Samples are sorted by an encoded `geometry_code`. Sorting is safe because the training loader is already shuffled.
- Samples sharing the same flip/rotation are transformed as one contiguous batch slice.
- Local and context RGB are concatenated into six channels so their identical transform is launched once per group.
- Mask and boundary are stacked as two target channels and transformed together.
- The final tensors are channels-last and ready for convolution.
- Muscle presence is calculated from the assembled mask batch with one reduction.
- Independent color-jitter parameters are applied in one batched CUDA function.

5.4 VRAM estimation

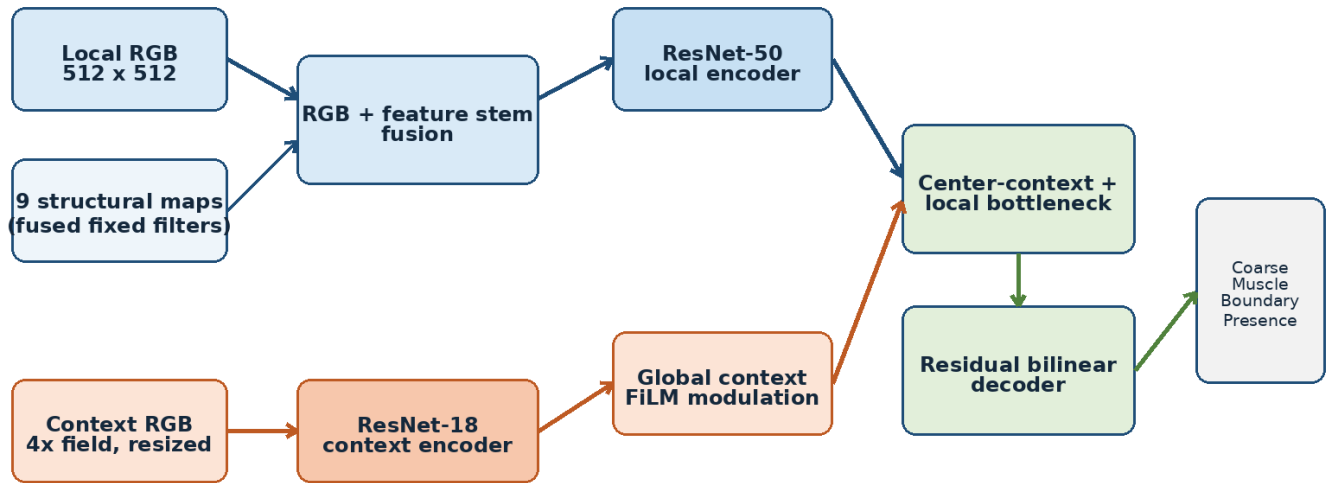
Before cache allocation, `train.py` estimates RGB, context, target, and optional feature-map storage. It subtracts `--cache-vram-reserve-gb` from currently free VRAM. The reserve must cover model parameters, optimizer state, activations, gradients, compilation workspaces, and transient kernels.

Recommended reserve

The code default is 7 GiB. The README example uses 10 GiB for a 20 GB RTX 4000 Ada. Use the larger reserve when enabling feature caching or increasing batch size, then measure actual peak memory.

6. Model architecture

AccurateTissueNet - current data and model flow



Final probabilities are mutually exclusive and sum to one. The coarse parent probability is split into Fibrocartilage and Muscle by a conditional child head.

Current revision: derivative filters are fused into two convolution calls; optional cached structural maps can bypass per-batch recomputation.

6.1 HandcraftedFeatures

The model computes nine native-resolution luminance-derived maps: Laplacian, Gaussian-weighted local deviation, gradient magnitude, structure-tensor maximum and minimum eigenvalues, coherence, Hessian determinant, and Hessian maximum and minimum eigenvalues.

The current revision fuses fixed operations:

- Sobel X, Sobel Y, Laplacian, Dxx, Dyy, and Dxy are stacked into one six-output convolution.
- Gray, gray squared, gx squared, gy squared, and gx*gy are smoothed by one grouped Gaussian convolution.
- Fixed scaling and asinh compression replace tile-dependent feature standardization, avoiding visible inference seams.
- The fused kernel banks are non-persistent buffers, preserving checkpoint compatibility.

6.2 Local encoder and feature stem

A pretrained ResNet-50 provides the local encoder. Its first RGB stem is fused with a learned stem operating on the nine structural maps. The fused stem then feeds the ResNet-50 residual stages and skip connections.

6.3 Context encoder and FiLM modulation

A pretrained ResNet-18 encodes the context field. Global pooled context produces FiLM gamma and beta values that modulate the local bottleneck. A center crop from the context feature map is resized and projected into the local bottleneck spatial size, then concatenated for fusion.

6.4 Decoder and heads

- Four residual decoder stages use bilinear interpolation, 1x1 projection, skip concatenation, GroupNorm, and GELU.
- The coarse head predicts five parent classes: Bone, Fibrocartilage-or-Muscle, Cartilage, Marrow, and Background.
- The Muscle child head predicts Muscle conditional on the parent probability.
- The boundary head predicts the Bone-Fibro interface.
- The regional Muscle-presence head pools local and context features and adds a learned prior to the dense Muscle logit.

6.5 Probability reconstruction

```
parent = P(Fibrocartilage_or_Muscle)
P(Fibrocartilage) = parent * (1 - P(Muscle | parent))
P(Muscle)         = parent * P(Muscle | parent)
```

The final six channels remain mutually exclusive and sum to one.

7. Losses, metrics, and optimization

7.1 Composite objective

Component	Weight	Purpose
Coarse cross-entropy	0.40	Learns the five parent tissue classes.
Conditional Muscle BCE	0.20	Separates Muscle from Fibrocartilage inside the parent region.
Six-class Dice	0.25	Improves overlap and class balance in final probabilities.
Boundary BCE + Dice	0.10	Sharpens the Bone-Fibro interface.
Regional Muscle presence BCE	0.05	Supplies a context-aware prior for Muscle occurrence.

Dice reductions and sensitive loss operations are accumulated in float32 under AMP to avoid FP16 reduction overflow at 512 x 512 resolution.

7.2 Metrics without per-batch synchronization

The current Metrics object keeps the confusion matrix and boundary TP/FP/FN counters on the active device. Loss components are accumulated as tensors and moved to CPU only at epoch end. This removes repeated `.item()` and `.cpu()` synchronization points from the inner loop.

7.3 Optimizer and precision path

- AdamW uses separate encoder and new-head learning rates: 3e-5 and 3e-4 by default.
- Fused CUDA AdamW is enabled unless `--no-fused-optimizer` is supplied.
- AMP and GradScaler are enabled by default on CUDA.
- TF32 is enabled for remaining FP32 matrix/convolution work unless `--no-tf32` is supplied.
- `channels_last` and cuDNN benchmark are enabled for CUDA convolution workloads.
- Gradient clipping defaults to 1.0.
- ReduceLROnPlateau monitors validation mIoU.

7.4 torch.compile

Use `--compile-model` to compile the model after checkpoint restoration. The default compile mode is `reduce-overhead`. The first training steps include compilation time, so compare steady-state patches/s after warm-up.

Mode	Use
default	Balanced compilation and performance.
reduce-overhead	Preferred current default for repeated fixed-shape training; performed well on the workstation.
max-autotune	More aggressive tuning; may print an SM-count warning and skip GEMM autotuning on this GPU.

8. Validation splitting

8.1 External specimen-level validation

For final scientific results, use `--val-data` pointing to a dataset split by patient or specimen. This is the strongest protection against leakage from neighboring tissue sections or acquisition-specific image characteristics.

8.2 Automatic patch-level split

When `--val-data` is absent, `split-unit` defaults to `patch`. A deterministic candidate pool is built for each source image, and a seeded permutation assigns validation patch IDs. The exact split is stored in the checkpoint under `data_split`.

```
python train.py --data /data/tissue/all --split-unit patch --patch-pool-per-image 8 --  
validation-fraction 0.15 ...
```

Interpretation warning

Patch-level validation is convenient for iteration and debugging but usually optimistic because training and validation patches may come from the same source images. Do not treat it as a final specimen-generalization estimate.

8.3 Image-level fallback

Use `--split-unit image` to restore the previous automatic whole-image split. This allows boundary/class-focused training sampling, but an external specimen/patient validation set remains preferable.

9. Recommended training workflows

9.1 Conservative baseline

```
python train.py --data /data/tissue/train --val-data /data/tissue/validation --device cuda
--crop-size 512 --context-scale 4 --batch-size 4 --epochs 80 --workers 4 --output
best_model.pt --last-output last_model.pt
```

This path uses the conventional CPU loader and is the easiest correctness baseline.

9.2 Cached RGB/context path

```
python train.py ... --cache-training-images --cache-vram-reserve-gb 10 --compile-model --
compile-mode reduce-overhead
```

This is the recommended performance configuration when the cache estimate fits comfortably. Training workers automatically become zero and the custom CUDA collate path is enabled.

9.3 Optional structural-feature cache

```
python train.py ... --cache-training-images --cache-handcrafted-features --cache-feature-tile-
size 1024 --cache-vram-reserve-gb 10 --compile-model --compile-mode reduce-overhead
```

Use only when the extra 18 bytes/source pixel fit. If cache construction runs out of memory, reduce `--cache-feature-tile-size`; this reduces temporary construction memory, not the final cache size.

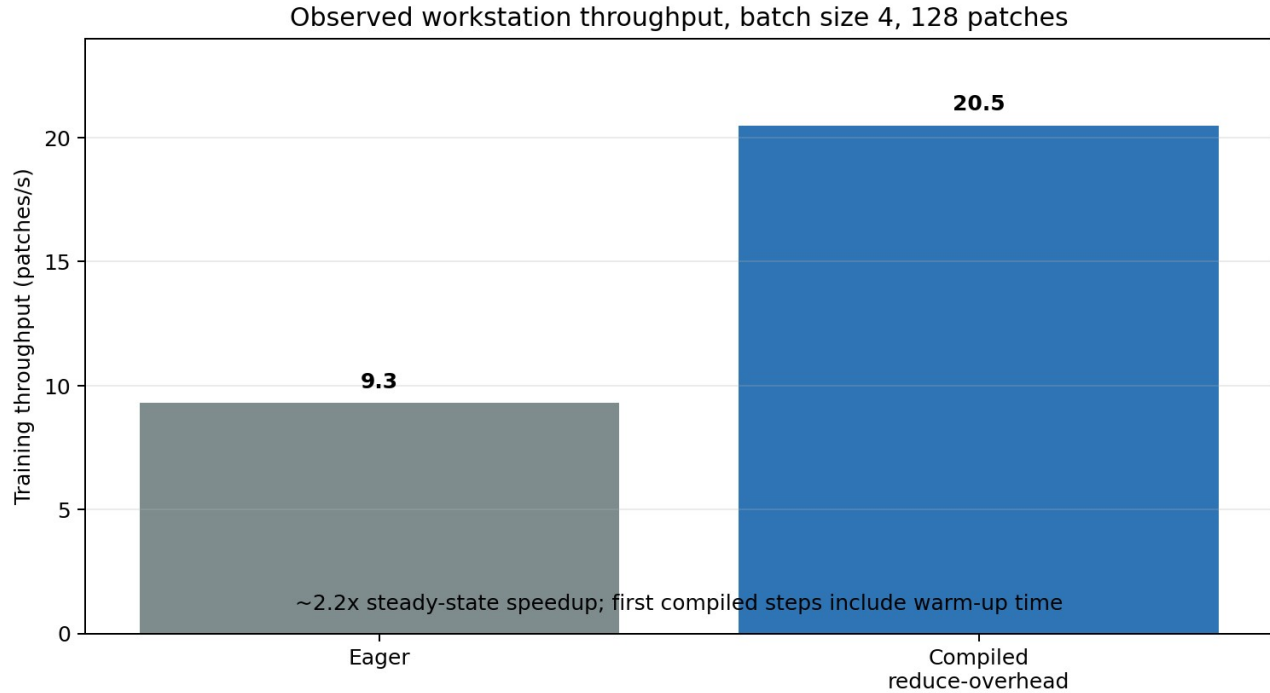
Validation requirement

Compare final area error and mIoU with and without cached structural maps. Feature caching changes the relation between photometric jitter and the structural maps, so it is an experimental option rather than an unconditional improvement.

9.4 Resume

```
python train.py ... --resume last_model.pt
```

10. Performance measurement and tuning



The chart records the current workstation observation supplied during development. It is not a general benchmark and should be reproduced after every major code, driver, CUDA, or PyTorch change.

10.1 Measure patches/s, not iterations/s

One optimizer iteration processes `batch_size` patches. Therefore, iterations/s is not comparable across batch sizes. The epoch logger synchronizes CUDA around the training phase and reports:

```
patches_per_second = len(train_dataset) / training_seconds
```

10.2 Batch-size experiments

Test batch sizes 4, 8, 12, and 16 with `accumulation_steps=1`. Record patches/s, peak VRAM, validation mIoU, area error, and optimizer updates per epoch. A lower iterations/s value can still be faster if batch size is larger.

Configuration	Throughput calculation	Optimizer updates
batch 4, 128 patches	iterations/s x 4	32 updates/epoch
batch 8, 128 patches	iterations/s x 8	16 updates/epoch
batch 16, 128 patches	iterations/s x 16	8 updates/epoch

10.3 Keep enough updates

The script calculates optimizer updates as `ceil(number_of_batches / accumulation_steps)` and warns below 25. A larger batch reduces the number of weight updates even when every patch is still seen.

```
optimizer_updates_per_epoch ~= total_patches /
    (batch_size * accumulation_steps)
```

If a larger batch improves throughput but produces too few updates, increase `--train-patches-per-image` or the total epoch count, or reduce the batch. Retune the learning-rate schedule if the effective optimization batch changes substantially.

10.4 Benchmark protocol

- Run at least one warm-up epoch when compilation is enabled.
- Use identical patch count, split, seed, AMP setting, and validation data.
- Compare the median of several steady-state epochs, not the first compiled epoch.

- Record patches/s and validation quality together. Speed without stable accuracy is not an improvement.
- Use `nvidia-smi dmon -s pucm` on Linux. In Windows Task Manager, select a CUDA or Compute graph rather than the default 3D engine.

11. Warnings and troubleshooting

Message / symptom	Meaning	Action
Not enough SMs to use max_autotune_gemm mode	TorchInductor skipped one aggressive GEMM autotuning path because the GPU does not meet its SM threshold. Compilation can still work.	Prefer reduce-overhead or default. Do not force internal autotune settings.
[0/0] prefix	Internal graph/frame and compile-instance identifier. It is not zero progress or zero GPU use.	No action unless repeated recompilations appear.
Fewer than 25 optimizer updates per epoch	The effective batch is large relative to the patch count, so weights change infrequently.	Increase patches per image, train longer, reduce batch/accumulation, or retune learning rate.
First compiled steps are very slow	torch.compile is tracing and generating kernels.	Measure later steady-state epochs.
Repeated compile pauses	New shapes or graph guards may trigger recompilation.	Set TORCH_LOGS=recompiles and inspect changing shapes/control flow.
CUDA OOM during cache construction	The final cache or temporary feature tile exceeds free memory.	Increase reserve, disable feature cache, lower feature tile size, or reduce batch.
Low reported GPU utilization but improving patches/s	Sampling tools may miss short compute bursts or show the wrong engine.	Use patches/s and validation quality as primary metrics.
Ethernet activity despite local files	May originate outside the loader or from storage redirection/synchronization.	Cached training should stop repeated image reads; compare network activity after startup.

Compile-time expectation

Cold compilation can take tens of seconds to several minutes for this multi-branch segmentation model. More than roughly 5-10 minutes without a first step, or repeated multi-minute pauses every epoch, merits recompilation logging and a fallback comparison with eager mode.

12. Prediction and physical-area quantification

12.1 Native-resolution prediction

```
python predict.py --image specimen.tif --checkpoint best_model.pt --output-dir prediction --  
device cuda --tile-size 512 --overlap 128 --batch-size 2
```

Prediction uses overlapping native-resolution tiles and center-weighted blending. Larger inference tiles can provide more detail per forward pass but require measured VRAM and do not replace adequate native-resolution training crops.

Outputs

- class_ids.png
- color_mask.png
- Six grayscale class-probability PNGs
- bone_fibro_boundary.png
- probabilities.npy with shape [6, height, width]
- classes.txt

probabilities.npy can be memory-mapped with `numpy.load(..., mmap_mode="r")`. Use `--no-save-probabilities` when only rendered PNGs are needed.

12.2 Area quantification

```
python quantify.py --prediction-dir prediction --pixel-width-um 0.50 --pixel-height-um 0.50
```

area_summary.csv reports hard-mask and probability-weighted areas in mm². Preserve acquisition pixel spacing and magnification metadata. Do not add morphology unless class-specific area bias has been measured.

13. File-by-file reference

data.py

The data system supports both a conventional CPU path and a CUDA-resident cache path.

- `discover_samples` validates matched images and PNG masks.
- `_extract_centered` and `_extract_centered_tensor` implement padded native crops.
- `estimate_training_cache_bytes` reports RGB, context, feature, target, and total VRAM.
- `_precompute_handcrafted_features` uses tiled evaluation with a two-pixel halo.
- `SegmentationDataset` caches RGB, context pyramid, device targets, boundary metadata, and optional features.
- `__getitem__` returns either ready CPU float tensors or deferred CUDA uint8 views plus transform metadata.

model.py

The model remains accuracy-first but now reduces fixed-feature kernel launches.

- `HandcraftedFeatures` uses fused derivative and grouped Gaussian banks.
- `AccurateTissueNet` accepts optional external `handcrafted_features`.
- `ResNet-50` handles local detail; `ResNet-18` handles context.
- `Context FiLM` modulates the local bottleneck before decoder fusion.
- Hierarchical probability reconstruction guarantees six mutually exclusive classes.

train.py

The training script owns the high-throughput batch path and experiment controls.

- `device_channels_last_collate` performs grouped geometry and batched target conversion.
- `batched_color_jitter` applies per-sample photometric parameters to a full batch.
- Metrics and loss accumulation remain on-device until epoch end.
- Patch-level splitting and split metadata improve reproducibility.
- TF32, fused AdamW, compilation, workload reporting, and patches/s are configurable.

predict.py and quantify.py

The deployment path preserves native resolution and physical calibration.

- Overlapping prediction tiles are blended into whole-image probability maps.
- Quantification converts pixel probabilities or hard labels to mm2 using calibrated pixel sizes.
- Final evaluation should include area error and boundary error, not only mIoU.

14. Revision checklist and suggested experiments

Implemented in the current repository revision

Item	Status
Fused fixed derivative/Gaussian filters	Implemented
VRAM RGB/context cache	Implemented
CUDA mask and boundary cache	Implemented
Optional FP16 handcrafted cache	Implemented
Batched channels-last CUDA collate	Implemented
Batched color jitter and Muscle presence	Implemented
Device-side metric accumulation	Implemented
Patch-level automatic split and checkpointed split metadata	Implemented
TF32 and fused AdamW defaults	Implemented
torch.compile modes	Implemented
patches/s and updates/epoch reporting	Implemented

Recommended next experiments

- Benchmark batch sizes 4, 8, 12, and 16 with compile mode reduce-overhead and accumulation_steps=1.
- Increase train-patches-per-image when a larger batch reduces updates below roughly 25-50 per epoch.
- Compare cached RGB/context alone against cached RGB/context plus handcrafted maps.
- Measure validation mIoU, per-class area error, and Bone-Fibro boundary error for each performance configuration.
- Profile whether patch-center sampling or context/feature cache construction remains material after compilation.
- Keep an eager, noncached reference configuration for regression testing.

Current observed baseline

At batch size 4 with 128 total training patches, eager mode produced approximately 9.3 patches/s and compiled reduce-overhead mode produced approximately 20.5 patches/s on the current workstation. Treat this as a local regression benchmark, not a universal target.

Source and revision note

This walkthrough was rebuilt from the current public repository revision at <https://github.com/PaladinMG/CNN-MalGen06> and the development observations recorded during the July 2026 optimization pass. When code and documentation differ, the repository source is authoritative.